SIDE A · THEORY. The question says … → go to: "O, Ω, Θ", "which is faster", "true or false", "order these" → ⋈ **1** · "T(n) = …", "solve", "from first principles", "recursion tree" → ↻ **2** · "sorted array", "O(log n)", "O(log² n)", "divide and conquer", "design an algorithm" → ⌕ **3** · "sort", "stable", "pivot", "k-th smallest", "median", "which sort would you use" → ⇅ **4** · **Side B:** "expected", "on average", "random" → ∑ **5** · "heap", "priority queue", "draw", "k-way", "largest k" → △ **6** · "lower bound", "at least … comparisons", "integers in range", "sort in O(n)" → ⊥ **7** · any formula → ✎ **8**. Boxes: ◉ meaning · § formal ▶ steps · ✎ answer shape ★ remember ⚠ pitfalls. Yellow-headed tables = patterns with worked examples.

1 · COMPARING GROWTH (A / B / U)

◉ **IN WORDS** Constant factors and small n never matter. O = "grows no faster than" (ceiling), Ω = "at least as fast" (floor), Θ = "same rate". **A** = algorithm A is faster (T_A grows slower), **B** = B faster, **U** = cannot tell from the bounds given.

§ **FORMAL, write exactly** $f = O(g)$: $\exists c > 0$, n_0 with $f(n) \le c\,g(n)$ for all $n \ge n_0$. $f = \Omega(g)$: $f(n) \ge c\,g(n)$ for all $n \ge n_0$. $f = \Theta(g)$: both (prove each side). Worst case $\Theta(g)$: $O(g)$ on every input **and** some input family costs $\Omega(g)$.

▶ **HOW TO** ① Keep only the biggest term on each side (solve recurrences first, section 2). ② Find the matching row below, apply its trick. ③ **Ratio test**, look at f/g as $n \to \infty$: $\to 0 \Rightarrow f = O(g)$, f slower · $\to$ a constant $\Rightarrow f = \Theta(g)$, same speed · $\to \infty \Rightarrow f = \Omega(g)$, f faster. ④ Smaller $T \Rightarrow$ that algorithm wins (**A** or **B**); same $\Theta \Rightarrow$ "same"; only O / Ω given ⇒ last row.

Shapes you see	Trick, then a worked example → answer
n^a vs n^b, maybe with logs	Compare exponents first; any n^ε beats any $(\log n)^k$. $n^{2.1}$ vs $n^2 \log^3 n$: $n^{0.1}$ beats $\log^3 n \Rightarrow$ **B**.
Ratios, products, close exponents	Take B/A: $\to \infty \Rightarrow$ **A** faster; $\to 0 \Rightarrow$ **B**; constant ⇒ same. $n^{1.5}$ vs $n^2/\sqrt{\log n}$: $B/A = \sqrt{n/\log n} \to \infty \Rightarrow$ **A**.
Exponentials c^n, $a^{\log_b n}$	Compare bases: $2^{n/2} = (\sqrt{2})^n$; $4^n = 2^{2n} \neq \Theta(2^n)$; $2^{n+5} = \Theta(2^n)$; $a^{\log_b n} = n^{\log_b a}$. 5^n vs $7^n \Rightarrow$ **A**. $n\,2^n$ vs $3^n \Rightarrow$ **A** ($(3/2)^n$ beats n). n^2 vs $3^{\log_4 n} = n^{0.79} \Rightarrow$ **B**.
n inside an exponent, $(\log n)^{\log n}$, $n^{\log\log n}$	Take log of both sides (logs keep order) and compare those; or write both as $2^{h(n)}$. n^9 vs $2^{\sqrt{n}}$: $9\log n$ vs $\sqrt{n} \Rightarrow$ **A**. $(\log n)^{\log n}$ vs $n^{\log\log n}$: both logs $= \log n \cdot \log\log n \Rightarrow$ same.
$\sqrt{\log n}$, $\log\sqrt{n}$, $\log\log n$	Substitute $m = \log n$ (or $m = \sqrt{n}$) and compare simple functions of m. $\sqrt{\log n}$ vs $\log\sqrt{n} = \frac{1}{2}\log n$: $\sqrt{m}$ vs $m/2 \Rightarrow$ **A**.
$n!$, n^n, $\log(n!)$	$\log(n!) = \Theta(n\log n)$; $(n/2)^{n/2} \le n! \le n^n$; $2^n = o(n!)$. 2^n vs $n! \Rightarrow$ **A**. $\log_3(n!)$ vs $n\log n \Rightarrow$ same.
Only O / Ω / Θ bounds given	Decided only if one side's **upper** bound is strictly below the other's **lower** bound. O vs O, Ω vs Ω, oscillating ($n^{\sin n}$) ⇒ **U**. $O(n^2)$ vs $\Omega(2^n) \Rightarrow$ **A**. $\Omega(n^3)$ vs $O(n^{2.81}) \Rightarrow$ **B**. $O(n^2)$ vs $O(n^{2.3}) \Rightarrow$ **U**. $O(\log n)$ vs $\Theta(\log\log n) \Rightarrow$ **U**.

Growth ladder, slow → fast

$1 < \log\log n < \log n < (\log n)^k < n^{0.01} < n < n\log n < n^2 < n^k < 2^n < n! < n^n$

- **Sums by hand.** $\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2)$; $\sum i^c = \Theta(n^{c+1})$. Upper: n terms × biggest. Lower, **half trick**: keep the top $n/2$ terms, each $\ge (n/2)^c$.
- Geometric: see ★ in section 2. $H_n = 1 + \frac{1}{2} + \cdots + \frac{1}{n} = \Theta(\log n)$ (blocks of $1, 2, 4, \ldots$ terms, each between $\frac{1}{2}$ and 1).

2 · RECURRENCES

◉ **IN WORDS** "Time for n = (number of calls) × (time for the smaller size) + (work in this call)". In $T(n) = a\,T(n/b) + f(n)$: a = how many recursive calls you make, b = each call gets the input divided by b, $f(n)$ = the work you do yourself outside the calls (splitting and combining). Mergesort: two calls on $n/2$ plus n merging ⇒ $2T(n/2) + n$. Solving = add up the work of every row of the recursion tree (★ below).

▶ **HOW TO** "From first principles": ① draw the tree, write row i cost and the leaves (★ below); ② put $L = \log_b n$, do the sum (section 8); ③ write "leaves give …, rows sum to …, so $T(n) = \Theta(\ldots)$" and say what wins: leaves, equal rows, or root.

★ **TOTAL COST of $T(n) = aT(n/b) + f(n)$, $T(1) = 1$. $L = \log_b n$ levels**

Row i has a^i nodes of size $n/b^i \Rightarrow$ row cost $a^i f(n/b^i)$. Leaves: $a^L = n^{\log_b a}$, cost 1 each. $T(n) = n^{\log_b a} + \sum_{i=0}^{L-1} a^i f(n/b^i)$ = leaves + sum of rows.
Sums for the total ($i = 0, \ldots, L-1$): all rows equal $X \Rightarrow L \cdot X$ · geometric $\sum r^i = \frac{r^L - 1}{r - 1}$: $r < 1 \Rightarrow$ about a constant, $r > 1 \Rightarrow$ about r^L (last term) · arithmetic $\sum i = \frac{L(L-1)}{2}$ · key identity $a^{\log_b n} = n^{\log_b a}$.

Worked: $T(n) = 4T(n/4) + n\log_4 n$

Row i: $4^i \cdot \frac{n}{4^i}\log_4 \frac{n}{4^i} = n(L - i)$ (4^i cancels; $\log(a/b) = \log a - \log b$). Leaves $4^L = n$. $T(n) = n + n(L + (L-1) + \cdots + 1) = n + n\frac{L(L+1)}{2} = \Theta(n\log^2 n)$.

★ **REMEMBER Which case am I in?** Write $f(n) = n^c \log^k n$ and $d = \log_b a$ (the leaves exponent, so leaves cost n^d). Two tricks that simplify row i: $a^i \cdot \frac{n^c}{b^{ic}} = n^c \left(\frac{a}{b^c}\right)^i$ and $\log_b \frac{n}{b^i} = \log_b n - i$. Compare c with d (table). **Shortcut when** $f(n) = n$ (most exam recurrences, $c = 1$): just compare a with b. Row i costs $n\,(a/b)^i$, a geometric series. $a > b \Rightarrow$ rows grow, leaves win, $\Theta(n^{\log_b a}) \cdot a = b \Rightarrow$ every row costs n, $\Theta(n\log n) \cdot a < b \Rightarrow$ rows shrink, root wins, $\Theta(n)$.

Case	Rows do	Answer $\Theta(n^\alpha \log^\beta n)$	α	β
$c < d$	grow toward leaves	$\Theta(n^d)$, leaves win	d	0
$c = d$	all about equal	$\Theta(n^d \log^{k+1} n)$	d	$k+1$
$c > d$	shrink from root	$\Theta(n^c \log^k n)$, root wins	c	k

Recurrence	Θ	Recurrence	Θ
$T(n/2) + 1$	$\log n$	$T(n/2) + n$	n
$2T(n/2) + n$	$n\log n$	$2T(n/2) + 1$	n
$2T(n/2) + n\log n$	$n\log^2 n$	$2T(n/2) + n^2$	n^2
$T(n-1) + n$	n^2	$T(n/2) + \log n$	$\log^2 n$
$T(\sqrt{n}) + 1$	$\log\log n$	$T(\sqrt{n}) + \log n$	$\log n$
$T(\alpha n) + T(\beta n) + n$, $\alpha + \beta = 1$	$n\log n$	same, $\alpha + \beta < 1$	n

- $T(\sqrt{n})$: set $m = \log n$ ($n = 2^m$, $\sqrt{n} = 2^{m/2}$) ⇒ $S(m) = S(m/2) + f$, solve, substitute back.
- $T(n-1) + f(n)$ is just $\sum_{i \le n} f(i)$: $T(n-1) + \log n = \log(n!) = \Theta(n\log n)$. $2T(n-1) + 1$ doubles each step: $2^n - 1$ (Hanoi).

3 · BINARY SEARCH & DIVIDE AND CONQUER

◉ **IN WORDS** Split the input into smaller copies of the same problem, solve them recursively, combine. The required running time tells you the shape:

Target time	Shape to use
$O(\log n)$ on a sorted or monotone array	one binary search: halve, $O(1)$ work per step
$O(\log^2 n)$	$\log n$ binary searches, or a binary search inside a binary search
$O(\log k)$, $k \ll n$	gallop: double an index until you pass k, then binary search that range
$O(n\log n)$	two halves + $O(n)$ combine (mergesort shape), or sort then one pass
$O(n\log k)$	heap of size k (section 6), or recursion that stops at depth $\log k$
$O(n)$ expected / $O(n)$ with "integers in a range"	random pivot, recurse on one side (section 4) / counting or radix sort (section 7)

LowerBound(A[1..n], x) *first i: A[i] ≥ x*
```
l ← 1; r ← n+1
while l < r:
  m ← ⌊(l+r)/2⌋
  if A[m] < x: l ← m+1 else r ← m
return l   (n+1 = none)   O(log n)
"< x" → "≤ x" gives first A[i] > x
```

RotatedMax(A, l, r) *rotated sorted array*
```
if l = r: return l
m ← ⌊(l+r)/2⌋
if A[m] > A[m+1]: return m   (the drop)
if A[m] ≥ A[l]: RotatedMax(A, m+1, r)
else RotatedMax(A, l, m)   O(log n)
```

Gallop(A, x) *answer at a small position k*
```
i ← 1; while i ≤ n and A[i] < x: i ← 2i
binary search in A[i/2 .. min(i, n)]   O(log k)
```

Kadane(A[1..n]) *max subarray, 1 pass*
```
best ← −∞; cur ← 0
for each x in A:
  cur ← max(x, cur + x)
  best ← max(best, cur)
return best   O(n)
```

Problem	Idea in one line	Time
Mergesort; count inversions ($i < j$, $A[i] > A[j]$)	sort both halves, Merge with an ∞ sentinel; for inversions, when an item of the right half is copied out add the number of left-half items still waiting	$2T(n/2) + n = \Theta(n\log n)$
Max subarray (best buy/sell pair = same on daily differences)	best in left, best in right, best crossing the middle = best suffix of left + best prefix of right, two linear scans	$2T(n/2) + n = \Theta(n\log n)$; Kadane $O(n)$
Multiply n-bit integers	halves: 4 products $\Rightarrow \Theta(n^2)$; Karatsuba: middle term $= (a_1 + a_0)(b_1 + b_0) - a_1 b_1 - a_0 b_0 \Rightarrow 3$ products. Matrices: Strassen 7 block products $\Rightarrow n^{2.807}$	$3T(n/2) + n = \Theta(n^{1.585})$
k-th smallest of two sorted arrays	compare $A[k/2]$ with $B[k/2]$; the smaller side's first $k/2$ cannot hold the answer, drop them, $k \leftarrow k/2$; $k = 1$: min of heads	$\Theta(\log k)$

✎ **ANSWER SHAPE** Design answer: ① idea in two sentences; ② "Procedure Name(input) → output"; ③ steps or pseudocode; ④ name each black box with its cost ("Select, $\Theta(n)$"); ⑤ recurrence or sum, solved, final Θ; ⑥ one line why correct.

4 · SORTING & SELECTION

Partition(A, l, r) *pivot to its final spot*
```
x ← A[r]; i ← l−1    i = wall of items ≤ x
for j ← l to r−1:
  if A[j] ≤ x: i ← i+1; swap A[i] ↔ A[j]
swap A[i+1] ↔ A[r]
return i+1          Θ(n), in place
```

InsertionSort(A[1..n]) *slide each item left*
```
for j ← 2 to n:
  key ← A[j]; i ← j−1
  while i ≥ 1 and A[i] > key:
    A[i+1] ← A[i]; i ← i−1
  A[i+1] ← key        O(n + inversions)
```

Quicksort(A, l, r)
```
if l ≥ r: return
m ← Partition(A, l, r)
Quicksort(A, l, m−1); Quicksort(A, m+1, r)
random pivot: swap random A[l..r] into A[r]
worst Θ(n²), expected Θ(n log n), sec. 5
```

Merge(A, l, m, r) *zip two sorted halves*
```
L ← A[l..m] + [∞]; R ← A[m+1..r] + [∞]
i ← 1; j ← 1
for k ← l to r:
  if L[i] ≤ R[j]: A[k] ← L[i]; i ← i+1
  else A[k] ← R[j]; j ← j+1   Θ(n)
inversions: in "else" add (m−l+1) − (i−1)
```

Select(A, l, r, k) *k-th smallest*
```
swap a random A[l..r] into A[r]
m ← Partition(A, l, r); rank ← m − l + 1
if k = rank: return A[m]
if k < rank: Select(A, l, m−1, k)
else Select(A, m+1, r, k − rank)
expected O(n); median of medians O(n)
```

Mergesort(A, l, r)
```
if l ≥ r: return
m ← ⌊(l+r)/2⌋
Mergesort(A, l, m); Mergesort(A, m+1, r)
Merge(A, l, m, r)   2T(n/2)+n = Θ(n log n)
stable (ties take L first), O(n) space
```

Sort	Worst	Stable	Space	Pick it when
Insertion	n^2, best n	yes	$O(1)$	"almost sorted", few inversions: $O(n + I)$
Mergesort	$n\log n$	yes	$O(n)$	"stable" (equal keys keep order), linked lists
Quicksort	n^2 , exp. $n\log n$	no	$O(\log n)$	"in practice"; worst: sorted input, last pivot
Heapsort	$n\log n$	no	$O(1)$	"in place", guaranteed $n\log n$
Counting / Radix	$n + k$ / $d(n + k)$	yes	$O(n + k)$	"ints in $[0, k]$" / "below n^c", c passes

SIDE B · APPLICATIONS. The question says … → go to: "expected number of …", "on average", "how many … until", "probability" → **Σ 5** · "min-heap", "max-heap", "priority queue", "draw the heap after", "build-heap", "k sorted lists", "largest k", "running median", "sliding window" → **△ 6** · "any algorithm needs at least", "lower bound", "decision tree", "integers in the range", "sort in O(n)", "stable", "radix" → **⊥ 7** · logs, sums, Stirling, binomials, e limits → **✎ 8**. Side A: ⋈ 1 compare growth · ↻ 2 recurrences · ⌕ 3 search and divide and conquer · ⇅ 4 sorting and selection.

Σ 5 · EXPECTATION & RANDOMIZED ALGORITHMS

◉ IN WORDS Expected value = long-run average. To find "how many X happen on average", never compute it directly: split the count into yes/no questions ("does person i get their own hat?"); the average of a yes/no question is just its probability; the average of a sum is the sum of the averages, always, even when the questions depend on each other. Hats: n people × $\frac{1}{n}$ = 1.

§ FORMAL, write exactly Indicator: $X_i = 1$ if event i happens, else 0; $E[X_i] = \Pr[X_i = 1]$. Linearity: $E[X+Y] = E[X] + E[Y]$ for any random variables, independent or not; so $E[\sum X_i] = \sum \Pr[\text{event } i]$. Waiting time: success probability p per try ⇒ expected $1/p$ tries. Tail sum: $E[X] = \sum_{i \geq 1} \Pr[X \geq i]$. Randomized running time = expected time on the worst input.

Question says	Pattern	Worked example
"expected number of [things] that …"	one indicator per thing; add the probabilities	Own hat: $\Pr = \frac{1}{n}$ each ⇒ $E = 1$. Own hat or a better one: $\Pr = \frac{1}{n}$ ⇒ $E = \frac{n+1}{2}$.
"expected number of pairs that …"	one indicator per pair; $\binom{n}{2}$ pairs	Birthdays, k people, n days: $E = \binom{k}{2}\frac{1}{n} = \frac{k(k-1)}{2n}$; reaches 1 at $k \approx \sqrt{2n}$.
"how many boxes / tries until you have …"	stages; stage i succeeds with p_i per step; add $1/p_i$	Coupon collector, n types: with i owned, $p = \frac{n-i}{n}$ ⇒ $E = \sum_{i=0}^{n-1} \frac{n}{n-i} = nH_n = \Theta(n \log n)$. First $n/2$ types: every $p \geq \frac{1}{2}$ ⇒ $E \leq n$.
"expected number of distinct … after m draws"	indicator per type; $\Pr[\text{seen}] = 1 - (1 - \frac{1}{n})^m$	n draws, n types: $E = n(1 - (1 - \frac{1}{n})^n) \approx n(1 - \frac{1}{e}) \approx 0.63n$.
"best so far", "record", "hire", "new maximum"	$\Pr[i\text{-th is the best of the first } i] = \frac{1}{i}$	$E = \sum_{i=1}^{n} \frac{1}{i} = H_n = \Theta(\log n)$. Cards guessed with memory: $\frac{1}{n-i+1}$, also H_n. Hash collisions: $\frac{1}{m}$ each ⇒ $\frac{n}{m}$.
"expected comparisons / calls of a randomized algorithm"	indicator per pair compared (quicksort), or good / bad stages (Select)	Quicksort: $\Pr[z_i, z_j \text{ compared}] = \frac{2}{j-i+1}$ ⇒ $\Theta(n \log n)$. Select, random-probe search: a good step ($\Pr = \frac{1}{2}$) removes $\frac{1}{4}$ ⇒ $\sum 2(\frac{3}{4})^i n = O(n)$.
"repeat until it works"	success p per round ⇒ $1/p$ rounds × cost per round	Majority by try-and-check: $p > \frac{1}{2}$ ⇒ ≤ 2 rounds of $O(n)$.

✎ ANSWER SHAPE "Let $X_i = 1$ if [event i], else 0, for $i = 1..n$. $\Pr[X_i = 1] = \dots$ because [reason]. $X = \sum X_i$ is [the quantity]. By linearity of expectation $E[X] = \sum_i \Pr[X_i = 1] = \cdots = \Theta(\dots)$." Waiting times: "Stage i ends with probability p_i per step, so it lasts $1/p_i$ steps on average; total $\sum 1/p_i$."

⚠ PITFALLS Excluding yourself: $(1 - \frac{1}{n})^{n-1}$, not $(1 - \frac{1}{n})^n$ · never assume independence, sums do not need it · write the exact form when asked, then simplify with $H_n \approx \ln n$, $(1 - \frac{1}{n})^n \to \frac{1}{e}$.

△ 6 · HEAPS

◉ IN WORDS A min-heap is a binary tree filled level by level from the left, where every parent ≤ its children, so the smallest item is always on top (find-min $O(1)$); siblings are in no order. Delete or change a key: bubble up if smaller, sift down if larger, $O(\log n)$. It lives in a plain array: node i has children $2i$, $2i+1$ and parent $\lfloor i/2 \rfloor$; height $\lfloor \log_2 n \rfloor$. Max-heap: same with ≥.

Insert(H, x) *append, then bubble up*
```
n ← n+1; H[n] ← x; i ← n
while i > 1 and H[i] < H[⌊i/2⌋]:
  swap H[i] ↔ H[⌊i/2⌋]; i ← ⌊i/2⌋   O(log n)
```

SiftDown(H, i) *push a too-big item down*
```
while 2i ≤ n:
  c ← 2i                    left child
  if c+1 ≤ n and H[c+1] < H[c]: c ← c+1
  if H[i] ≤ H[c]: stop
  swap H[i] ↔ H[c]; i ← c   O(log n)
```

ExtractMin(H) *last item goes on top*
```
x ← H[1]; H[1] ← H[n]; n ← n−1
SiftDown(H, 1); return x    O(log n)
```

BuildHeap(A[1..n]) *fix from the bottom up*
```
for i ← ⌊n/2⌋ down to 1:
  SiftDown(A, i)    O(n), leaves skip
```

Heapsort(A[1..n]) *pull out the max n times*
```
BuildMaxHeap(A)
for i ← n down to 2:
  swap A[1] ↔ A[i]; n ← i−1; SiftDown(A, 1)
```
O(n log n), in place, not stable

Max-heap *flip every < and ≤*
Insert bubbles up while bigger than parent; SiftDown swaps with the larger child.

▶ HOW TO Drawing "insert these one by one" (0.5 off per mistake): for **every** insert ① write the array with the new number appended, ② compare with its parent at $\lfloor i/2 \rfloor$, swap while smaller, writing the array after each swap, ③ draw the tree. Check: bottom level filled from the left, every parent ≤ its children.

Insert 4, 3, 8, 2 into an empty min-heap, then extract-min

4: [4] · 3: [4, 3], $3 < 4$ swap ⇒ [3, 4] · 8: [3, 4, 8], $8 \geq 3$ stays · 2: [3, 4, 8, 2], parent of slot 4 is slot 2 (holds 4): swap ⇒ [3, 2, 8, 4]; parent of slot 2 is slot 1 (holds 3): swap ⇒ [2, 3, 8, 4]. Tree: 2 on top, children 3 and 8, 4 under 3. Extract-min: take 2; last item 4 goes on top ⇒ [4, 3, 8]; smaller child 3 ⇒ swap ⇒ [3, 4, 8].

★ REMEMBER Build-heap is $O(n)$: a node at height h costs $O(h)$ and at most $n/2^{h+1}$ nodes have height h, so total $\leq \sum_h h \cdot \frac{n}{2^{h+1}} = \frac{n}{2}\sum_h \frac{h}{2^h} = \frac{n}{2} \cdot 2 = O(n)$. Most nodes are near the bottom and barely move. Inserting one by one costs $\sum \log i = \Theta(n \log n)$.

Question says	Design	Time
"merge k sorted lists", N items	min-heap of k entries (value, list, position); pop the min, output, push the next item of that list	$O(N \log k)$, space $O(k)$
"each element at most k from its sorted place"	min-heap of the next $k+1$ items; pop, push the next	$O(n \log k)$
"largest (smallest) k of n"	min-heap of size k: push each item, pop the min when size exceeds k. Or Select the k-th in $O(n)$ then sort k items	$O(n \log k)$; $O(n + k \log k)$
"median after every insertion"	max-heap of the lower half, min-heap of the upper half, sizes differ ≤ 1; insert on the right side, move one root if unbalanced	insert $O(\log n)$, median $O(1)$
"maximum of every window of width W"	max-heap of (value, index); pop roots whose index left the window (lazy; heap may grow to n); with a position map delete the leaver at once, heap stays size W	$O(n \log n)$ lazy; $O(n \log W)$ with map

⊥ 7 · LOWER BOUNDS & LINEAR-TIME SORTING

◉ IN WORDS Each comparison is a yes/no question; q questions can tell apart at most 2^q cases. If a problem has N possible answers you need $2^q \geq N$, so $q \geq \log_2 N$. Sorting has $n!$ answers and $\log(n!) \approx n \log n$, so no comparison sort beats $n \log n$. Counting and radix sort never compare keys, so they escape, for small integer keys only.

▶ HOW TO Lower bound, four lines: ① count the distinct possible outputs N; ② "any comparison-based algorithm is a binary decision tree with at least N leaves"; ③ "a binary tree of height h has at most 2^h leaves, so $h \geq \log_2 N$"; ④ simplify; height = worst-case comparisons.

Question says	Outputs N	Bound
"sort n items"	$n!$	$\log_2 n! \geq \frac{n}{2}\log_2 \frac{n}{2} = \Omega(n \log n)$
"the k smallest in sorted order"	$\frac{n!}{(n-k)!} \geq (n/2)^k$	$\Omega(k \log n)$
"merge two sorted arrays of n"	$\binom{2n}{n} \approx 4^n/\sqrt{\pi n}$	$\approx 2n = \Omega(n)$
"split into k sorted blocks of n/k"	$n!/((n/k)!)^k$	$\Omega(n \log k)$

CountingSort(A[1..n], 0..k) *count, place*
```
C[0..k] ← 0
for j ← 1 to n: C[A[j]] ← C[A[j]] + 1
for v ← 1 to k: C[v] ← C[v] + C[v−1]
for j ← n down to 1:    backwards = stable
  B[C[A[j]]] ← A[j]; C[A[j]] ← C[A[j]] − 1
return B                Θ(n + k)
```

RadixSort(A, d digits) *stable pass per digit*
```
for digit ← rightmost to leftmost:
  CountingSort A by that digit only
```
ties keep their current order (stable)
Θ(d(n+k)); below n^c: c passes base n

Question says	What to do	Why / time
"integers in $[0, k]$", $k = O(n)$	counting sort	$\Theta(n + k)$
"integers below n^c" (e.g. n^3)	view each key as c digits in base n; radix sort, c passes	$c \times \Theta(n + n) = \Theta(n)$. Linear ⇔ range polynomial ⇔ keys have $O(\log n)$ bits. Range $n^{\log n}$ ⇒ $\log n$ passes ⇒ $\Theta(n \log n)$, not linear
keys are not integers (fractions p/q, $p, q \leq n$)	build an integer key that keeps the order: distinct fractions differ by $\geq 1/n^2$, so $K = \lfloor n^2 p/q \rfloor \in [n, n^3]$ preserves order; radix sort K	$\Theta(n)$. No gcd ($\log n$ each), no floats (rounding merges close values)
"why scan backwards", "why must each pass be stable"	backwards scan hands equal keys their slots right to left, so input order survives; radix is correct by induction on digits, and that step needs stability	forward scan reverses ties

✎ 8 · FORMULAS

Logs

$\log(ab) = \log a + \log b$
$\log(a^b) = b \log a$
$\log_b a = \frac{\log a}{\log b} = \frac{1}{\log_a b}$
$\log_c n = \Theta(\log_2 n)$
$\log(n!) = \Theta(n \log n)$
$\log_2 3 \approx 1.585$, $\log_2 7 \approx 2.807$, $\log_4 3 \approx 0.79$

Exponents

$a^{\log_b n} = n^{\log_b a}$
$2^{\log_2 n} = n$
$(c^m)^n = c^{mn}$, $c^m c^n = c^{m+n}$
$4^n = 2^{2n} \neq \Theta(2^n)$
$2^{n+c} = \Theta(2^n)$
$n^{1/\log n} = 2$

Sums

$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$
$\sum_{i=0}^{k} r^i = \frac{r^{k+1}-1}{r-1}$
$\sum_{i \geq 0} x^i = \frac{1}{1-x}$, $x < 1$
$\sum_{i \geq 1} i x^i = \frac{x}{(1-x)^2}$;
$\sum \frac{i}{2^i} = 2$
$H_n = \sum_{i=1}^{n} \frac{1}{i} \approx \ln n + 0.58$
$\sum_{i \leq n} \log i = \log(n!)$

Approximations

$n! \approx \sqrt{2\pi n}\,(n/e)^n$
$(n/2)^{n/2} \leq n! \leq n^n$
$\binom{n}{k} \leq \frac{n^k}{k!} \leq (\frac{en}{k})^k$
$\binom{2n}{n} \approx \frac{4^n}{\sqrt{\pi n}}$
$(1 - \frac{1}{n})^n \to \frac{1}{e} \approx 0.37$,
$(1 + \frac{1}{n})^n \to e$
$1 - x \leq e^{-x}$, so $(1 - p)^k \leq e^{-pk}$